WEJ LAB

Dimension Reduction Techniques

W. Evan Johnson, Ph.D.
Professor, Division of Infectious Disease
Director, Center for Data Science
Associate Director, Center for Biomedical Informatics and Health AI
Rutgers University – New Jersey Medical School

2026-10-28

RUTGERS HEALTH
Center for Data Science

WEJ LAB

Learning objectives

By the end of this lecture you should be able to:

- Explain PCA as a rotation that concentrates variance
- Choose how many PCs to keep, read loadings, and decide whether to scale
- Connect PCA, eigendecomposition, and the SVD
- Use PCA to spot **batch effects**
- Apply NMF, t-SNE, and UMAP, and know their pitfalls
- Run a basic single-cell workflow: PCA $\rightarrow$ graph clustering $\rightarrow$ UMAP

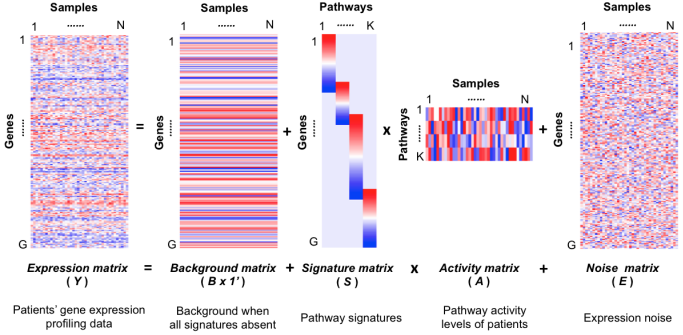
Dimension reduction

- Biomedical data often have many predictors (genes, taxa, proteins)
- Hard to visualize relationships in high dimensions
- **Dimension reduction:** fewer dimensions, while preserving key features such as distances between observations

Dimension reduction: motivation

Microbiome 16S data example:

```
library(animalcules)
run_animalcules()
```



Dimension reduction

- The engine behind it all: the **singular value decomposition (SVD)**
- We start with **principal component analysis (PCA)**
- First, a simple example to build intuition

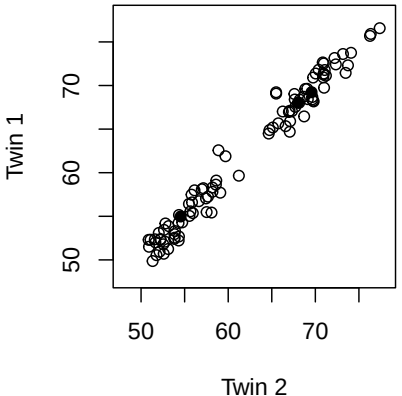
Dimension reduction: twin heights

Simulated heights for 100 pairs of twins (children and adults):

```
set.seed(1988)
library(MASS)
n <- 100
Sigma <- matrix(c(9, 9 * 0.9, 9 * 0.92, 9 * 1), 2, 2)
x <- rbind(mvrnorm(n / 2, c(69, 69), Sigma),
           mvrnorm(n / 2, c(55, 55), Sigma))
```

Dimension reduction: twin heights

Highly correlated, with two groups (adults and children):



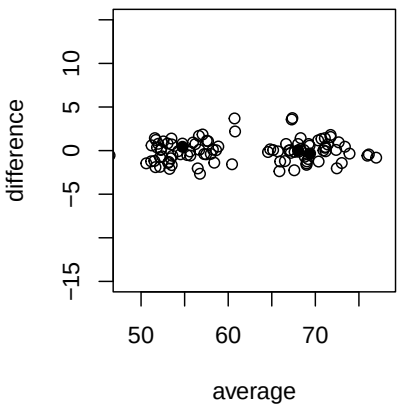
Dimension reduction: twin heights

- Pretend 2 dimensions is too many: can we reduce to 1 and still see the 2 groups?
- Distances between points mostly run along the diagonal
- Try the **average** and the **difference**:

```
z  <- cbind(average=(x[,2] + x[,1])/2,
            difference=x[,2] - x[,1])
```

Dimension reduction: twin heights

Most of the distance is explained by the first dimension, the average:



Linear transformations

We applied a linear transformation to each row:

$$Z_{i,1} = \frac{1}{2}X_{i,1} + \frac{1}{2}X_{i,2}, \qquad Z_{i,2} = X_{i,1} - X_{i,2}$$

In matrix form:

$$Z = XA \text{ with } A = \begin{pmatrix} 1/2 & 1 \\ 1/2 & -1 \end{pmatrix}$$

Linear transformations

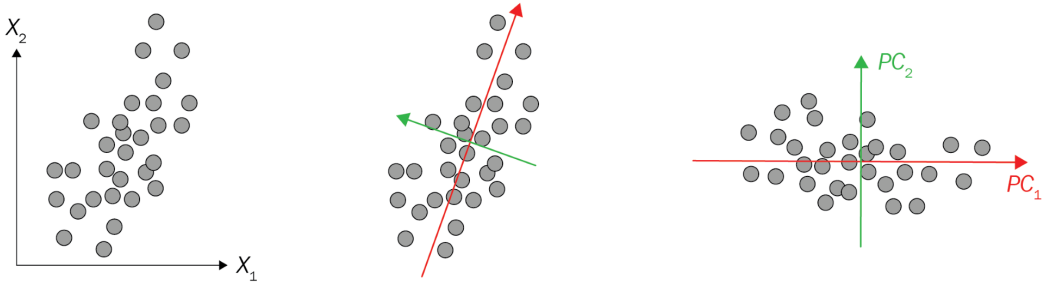
And we can transform back with $X = ZA^{-1}$:

```
A <- matrix(c(1/2,1/2,1,-1), nrow=2)
z <- x %*% A
x_back <- z %*% solve(A)
max(abs(x - x_back))
```

```
## [1] 1.421085e-14
```

Dimension reduction: find A that *rotates* the information in X into the first few columns of $Z = XA$, then keep only those columns.

Principal component analysis



Principal component analysis

- The **first principal component (PC)** is the orthogonal linear transformation of X with maximum variance
- Each next PC has maximum variance, orthogonal to the previous ones

```
pca <- prcomp(x)
pca$rotation
```

```
##                PC1        PC2
## [1,] -0.7022354  0.7119448
## [2,] -0.7119448 -0.7022354
```

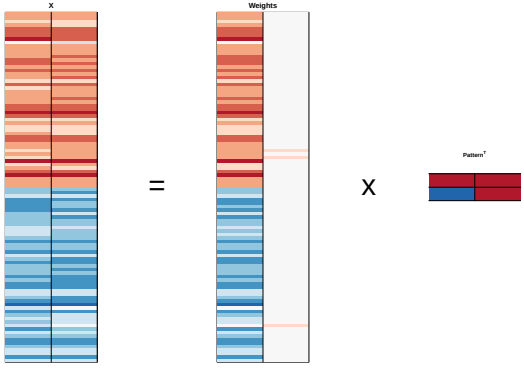
Principal component analysis

- PC1 is (almost) the average; PC2 is (almost) the difference
- `$rotation`: the transformation A (**loadings**)
- `$x`: the transformed data Z (**PC scores**)

```
head(cbind(z, pca$x), 3)
```

```
##                                    PC1         PC2
## [1,] 69.39688  0.3328979 -10.520283 0.28543334
## [2,] 68.01154 -0.0248272  -8.562911 0.01903868
## [3,] 65.96233  1.2495542  -5.658769 0.90024455
```

Principal component analysis



Principal component analysis

- Works for any number of columns p
- If the PCs after the first k have tiny variance, k dimensions approximate the distances well
- If $k \ll p$: an efficient **reduction** of the data

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Iris example

Four flower measurements for three species:

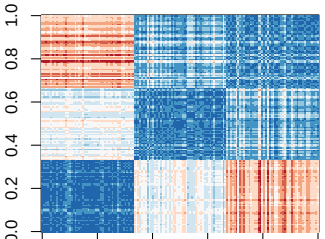
```
head(iris)
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1          5.1         3.5          1.4         0.2  setosa
## 2          4.9         3.0          1.4         0.2  setosa
## 3          4.7         3.2          1.3         0.2  setosa
## 4          4.6         3.1          1.5         0.2  setosa
## 5          5.0         3.6          1.4         0.2  setosa
## 6          5.4         3.9          1.7         0.4  setosa
```

Iris example

Distances between flowers: one species is very different from the other two

```
x <- iris[,1:4] %>% as.matrix()
d <- dist(x)
cols <- rev(RColorBrewer::brewer.pal(9, "RdBu"))
image(as.matrix(d), col = cols)
```



Iris example

Three of the four features are highly correlated:

```
cor(x)
```

```
##              Sepal.Length Sepal.Width Petal.Length Petal.Width
## Sepal.Length    1.0000000  -0.1175698    0.8717538   0.8179411
## Sepal.Width    -0.1175698   1.0000000   -0.4284401  -0.3661259
## Petal.Length    0.8717538  -0.4284401    1.0000000   0.9628654
## Petal.Width     0.8179411  -0.3661259    0.9628654   1.0000000
```

So PCA should capture these distances in about two dimensions.

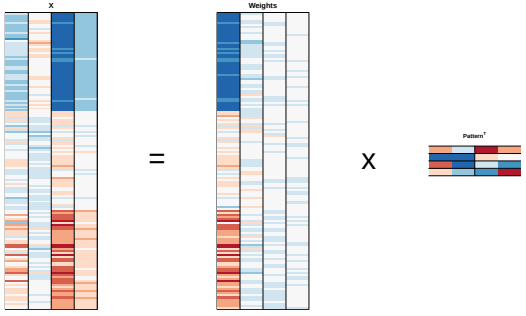
Iris example

```
pca <- prcomp(x)
summary(pca)
```

```
## Importance of components:
##                           PC1     PC2    PC3     PC4
## Standard deviation     2.0563 0.49262 0.2797 0.15439
## Proportion of Variance 0.9246 0.05307 0.0171 0.00521
## Cumulative Proportion  0.9246 0.97769 0.9948 1.00000
```

The first two PCs explain 97% of the variance.

Iris example

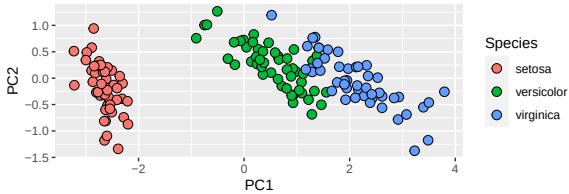


WEJ LAB

RUTGERS HEALTH
Center for Data Science

Iris example: PC1 vs. PC2

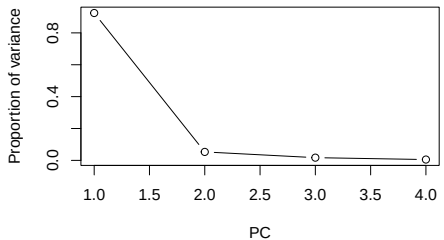
```
data.frame(pca$x[,1:2], Species=iris$Species) %>%
  ggplot(aes(PC1, PC2, fill = Species)) +
  geom_point(cex=3, pch=21) + coord_fixed(ratio = 1)
```



How many PCs? The scree plot

Plot the variance explained by each PC and look for where it levels off:

```
pve <- pca$sdev^2 / sum(pca$sdev^2)
plot(pve, type = "b", xlab = "PC", ylab = "Proportion of variance")
```



Loadings: what drives each PC?

The **loadings** (rotation) show how much each feature contributes to each PC:

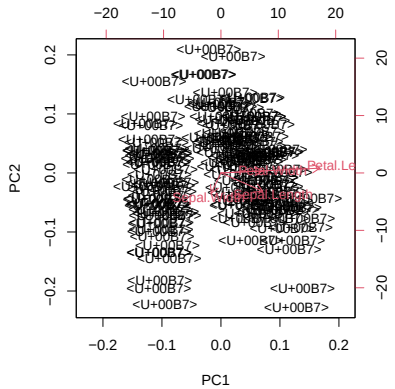
```
round(pca$rotation[, 1:2], 2)
```

```
##                PC1   PC2
## Sepal.Length  0.36 -0.66
## Sepal.Width  -0.08 -0.73
## Petal.Length  0.86  0.17
## Petal.Width   0.36  0.08
```

PC1 is mostly petal length; PC2 is mostly sepal size (length and width).

Loadings: the biplot

```
biplot(pca, xlabs = rep("·", nrow(x)), cex = 0.9)
```



Should you scale before PCA?

- **Covariance** PCA (default): high-variance features dominate
- **Correlation** PCA (`scale. = TRUE`): every feature counts equally
- Units matter! Petal length in millimeters instead of centimeters:

```
x_mm <- x; x_mm[, "Petal.Length"] <- 10 * x_mm[, "Petal.Length"]
round(prcomp(x_mm)$rotation[, 1], 2)
```

```
## Sepal.Length  Sepal.Width Petal.Length  Petal.Width
##         0.04        -0.01         1.00         0.04
```

```
round(prcomp(x_mm, scale. = TRUE)$rotation[, 1], 2)
```

```
## Sepal.Length  Sepal.Width Petal.Length  Petal.Width
##         0.52        -0.27         0.58         0.56
```

Eigenvectors and eigenvalues

For a square matrix $\mathbf{A}$, an **eigenvalue** λ and **eigenvector** $\mathbf{x} \neq 0$ satisfy

$$\mathbf{A}\mathbf{x} = \lambda\mathbf{x}$$

- Equivalently, $\det(\mathbf{A} - \lambda\mathbf{I}) = 0$ (the **characteristic equation**)
- *Eigen-* is German for "own" or "characteristic"

Eigenvalue decomposition

Put the eigenvalues on the diagonal of Λ and the eigenvectors in the columns of $\mathbf{Q}$. Then $\mathbf{AQ} = \mathbf{Q}\Lambda$, and if $\mathbf{Q}$ is invertible:

$$\mathbf{A} = \mathbf{Q}\Lambda\mathbf{Q}^{-1}$$

Example: matrix powers become scalar powers, $\mathbf{A}^p = \mathbf{Q}\Lambda^p\mathbf{Q}^{-1}$.

Eigenvectors and PCA

PCA of an $N \times p$ data matrix $\mathbf{Y}$ is the eigendecomposition of its **covariance matrix**. A covariance matrix is symmetric, so $\mathbf{Q}$ is orthogonal ($\mathbf{Q}^{-1} = \mathbf{Q}^{\top}$):

$$Cov(\mathbf{Y}) = \mathbf{Q}\Lambda\mathbf{Q}^{\top}$$

- Eigenvectors (columns of $\mathbf{Q}$): the **loadings** (rotation)
- Eigenvalues: the **variance** explained by each PC
- Applying the rotation to $\mathbf{Y}$ gives the **PC scores**

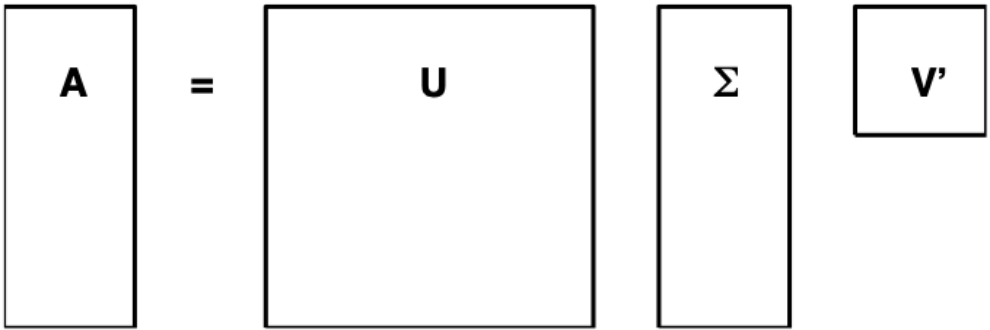
Singular value decomposition

The **SVD** factors **Y** directly (no covariance matrix needed). For an $N \times p$ matrix with $p < N$:

$$Y = UDV^\top$$

- U $(N \times p)$ and V $(p \times p)$ are **orthogonal**
- D is **diagonal**, with $d_{1,1} \geq d_{2,2} \geq \cdots \geq d_{p,p}$

Singular value decomposition



WEJ LAB
RUTGERS HEALTH
Center for Data Science

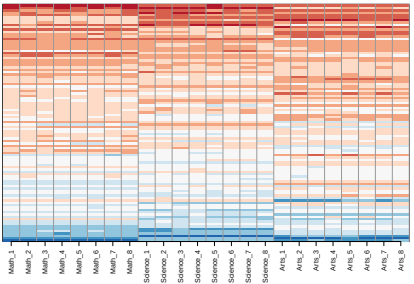
SVD example: test scores

Scores for 100 students on 24 tests (0 = average, +25 = A, -25 = F):

```
set.seed(1987)
n <- 100
k <- 8
Sigma <- 64  * matrix(c(1, .75, .5, .75, 1, .5, .5, .5, 1), 3, 3)
m <- MASS::mvrnorm(n, rep(0, 3), Sigma)
m <- m[order(rowMeans(m), decreasing = TRUE),]
y <- m %x% matrix(rep(1, k), nrow = 1) +
  matrix(rnorm(matrix(n * k * 3)), n, k * 3)
colnames(y) <- c(paste(rep("Math",k), 1:k, sep="_"),
  paste(rep("Science",k), 1:k, sep="_"), paste(rep("Arts",k), 1:k, sep="_"))
```

SVD example: test scores

Are all students about equally good? Does doing well in one subject predict another?



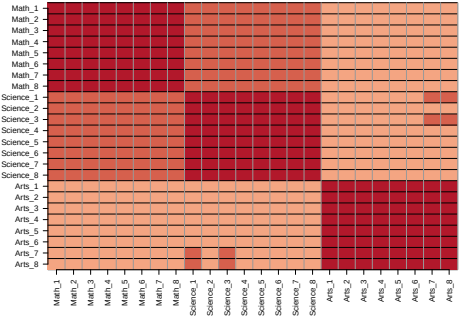
SVD example: test scores

How would you describe the data?

a. The test scores are all independent of each other.
b. Students who test well are at the top, and there seem to be three groupings by subject.
c. Students good at math are not good at science.
d. Students good at math are not good at the arts.

WEJ LAB
RUTGERS HEALTH
Center for Data Science

SVD example: correlations



SVD example: correlations

Which best describes what you see?

a. The test scores are independent.
b. Math and science are highly correlated, but the arts are not.
c. High correlation within a subject, none across subjects.
d. All tests are correlated, more so for math and science, and most within each subject.

WEJ LAB
RUTGERS HEALTH
Center for Data Science

SVD in R

`svd` returns U, V, and the diagonal of D:

```
s <- svd(y)
names(s)
```

```
## [1] "d" "u" "v"
```

```
dim(s$u); dim(s$v); length(s$d)
```

```
## [1] 100  24
```

```
## [1] 24 24
```

```
## [1] 24
```

WEJ LAB

RUTGERS HEALTH
Center for Data Science

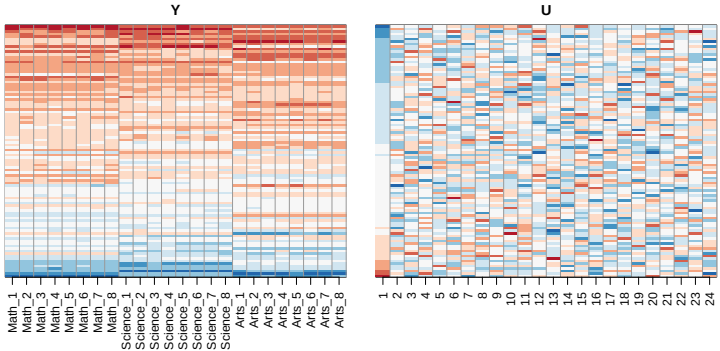
SVD in R

Check that we recover Y:

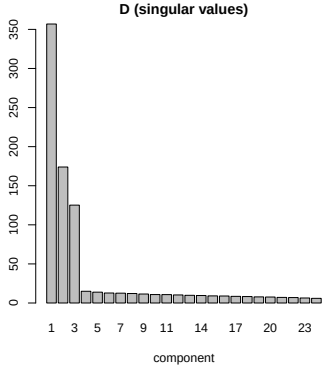
```
y_svd <- s$u %*% diag(s$d) %*% t(s$v)
max(abs(y - y_svd))
```

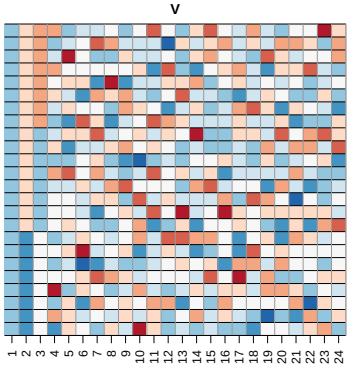
```
## [1] 5.329071e-14
```

Visualizing the SVD: Y and U



Visualizing the SVD: D and V





Variance explained

d_j^2 is proportional to the variance explained by component j:

```
round(s$d^2/sum(s$d^2), 3)[1:6]
```

```
## [1] 0.726 0.172 0.089 0.001 0.001 0.001
```

```
round(cumsum(s$d^2)/sum(s$d^2), 3)[1:6]
```

```
## [1] 0.726 0.898 0.988 0.989 0.990 0.991
```

Three components (math, science, arts) explain almost all of the variance.

PCA and the SVD are the same thing

For **column-centered** Y with SVD $Y = UDV^\top$:

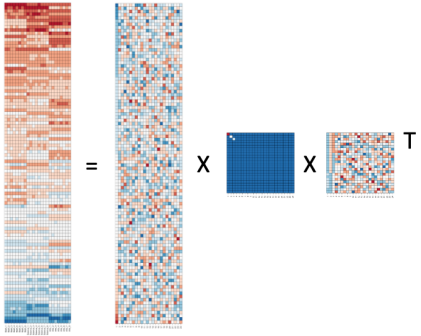
- Loadings (`$rotation`) $= V$
- PC scores (`$x`) $= UD$
- PC variances $= d_j^2/(N-1)$

```
yc <- scale(y, center = TRUE, scale = FALSE)
s_c <- svd(yc); p_y <- prcomp(y)
# equal up to sign:
max(abs(abs(p_y$x) - abs(s_c$u %*% diag(s_c$d))))
```

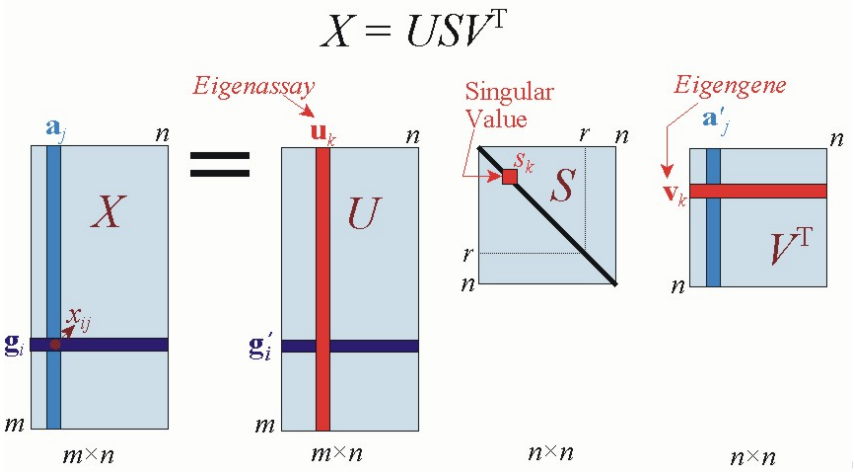
```
## [1] 7.81597e-14
```

`prcomp` actually computes the SVD; it never forms the covariance matrix.

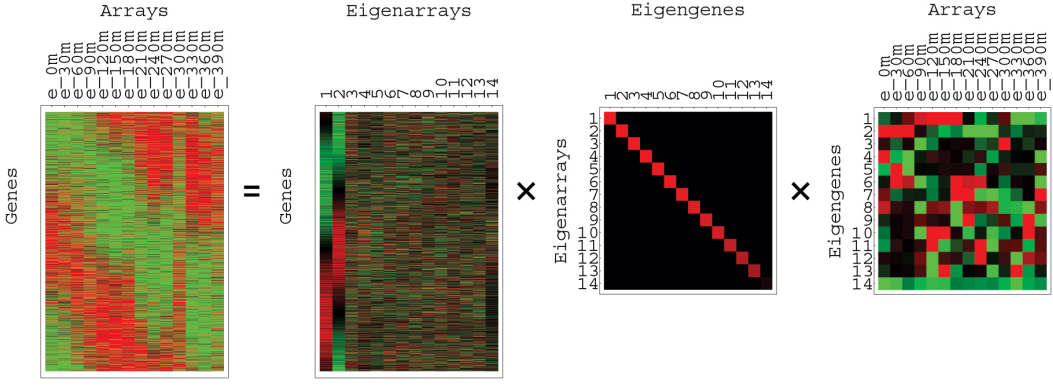
Putting it together



Singular value decomposition



SVD of gene expression: “eigengenes”



Batch effects

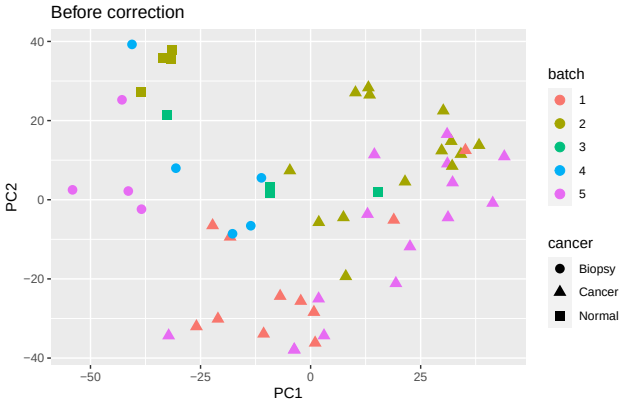
- **Batch effects:** technical variation from processing date, lab, reagent lot, or platform
- Often the **largest** source of variation in omics data
- Can hide biology, or create false findings if confounded with the outcome
- PCA is the first diagnostic: color the PC plot by batch

Batch effects: bladder cancer data

57 bladder samples (normal, cancer, biopsy) processed in 5 batches (`bladderbatch`, Bioconductor):

```
library(bladderbatch); library(sva)
data(bladderdata)
e  <- exprs(bladderEset); pd <- pData(bladderEset)
top <- order(apply(e, 1, var), decreasing = TRUE)[1:2000]
pc_plot <- function(m, main) {
  p <- prcomp(t(m[top, ]))
  data.frame(p$x[, 1:2], batch = factor(pd$batch), cancer = pd$cancer) %>%
    ggplot(aes(PC1, PC2, color = batch, shape = cancer)) +
    geom_point(size = 3) + ggtitle(main)
}
pc_plot(e, "Before correction")
```

Batch effects: bladder cancer data



Samples group by batch as much as by cancer status.

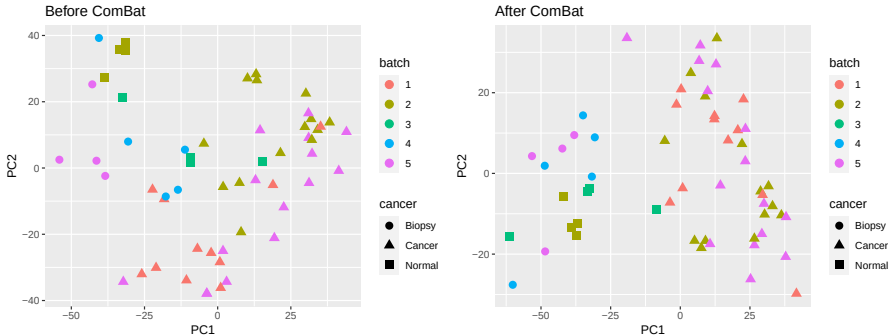
Batch correction with ComBat

ComBat (Johnson, Li & Rabinovic, 2007) removes batch-specific location and scale shifts, borrowing strength across genes (empirical Bayes). Protect the biology with `mod`:

```
mod <- model.matrix(~ cancer, data = pd)
e_cb <- ComBat(dat = e, batch = pd$batch, mod = mod)
```

For RNA-seq counts, use **ComBat-seq** (`sva::ComBat_seq`).

Batch correction with ComBat



Non-negative matrix factorization (NMF)

Like SVD, a linear factorization, but **all entries non-negative**:

$$\mathbf{X} \approx \mathbf{WH}$$

- $\mathbf{X}$: $N \times P$ data
- $\mathbf{W}$: $N \times K$ (basis / sample weights)
- $\mathbf{H}$: $K \times P$ (components / feature loadings)

NMF: interpretation

- **Additive, parts-based:** components only add, never cancel
- PCA/SVD components mix positive and negative values ("holistic")
- Natural for non-negative data: counts, intensities, expression levels

WEJ LAB
RUTGERS HEALTH
Center for Data Science

NMF: fitting

Minimize a loss, e.g.:

- **Frobenius norm:** $\frac{1}{2}\sum_{i,j}(X_{ij} - (WH)_{ij})^2$
- **KL divergence:** for count data (Poisson likelihood)

Non-convex: results depend on the starting values, so run it several times (`nrun`) and keep the best.

WEJ LAB
RUTGERS HEALTH
Center for Data Science

NMF: multiplicative updates

Lee & Seung (2001): multiply instead of subtract, so entries stay non-negative:

$$H_{a\mu} \leftarrow H_{a\mu} \frac{(\mathbf{W}^T\mathbf{X})_{a\mu}}{(\mathbf{W}^T\mathbf{W}\mathbf{H})_{a\mu}} \qquad W_{i\alpha} \leftarrow W_{i\alpha} \frac{(\mathbf{X}\mathbf{H}^T)_{i\alpha}}{(\mathbf{W}\mathbf{H}\mathbf{H}^T)_{i\alpha}}$$

The loss never increases, so the algorithm converges.

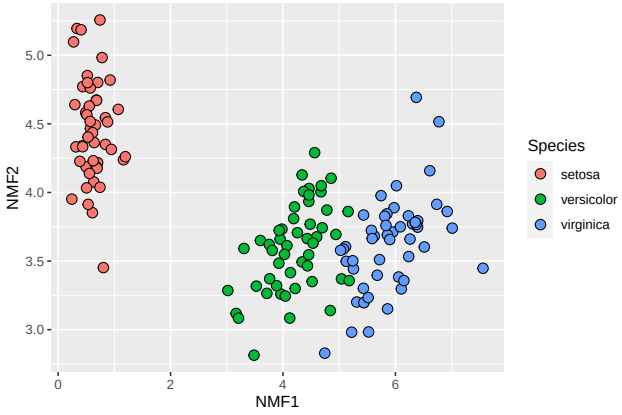
When to use NMF

- Data must be **non-negative**
- You want **interpretable, additive parts**. Examples:
  - **Topic modeling:** word counts $\rightarrow$ topics
  - **Images:** faces $\rightarrow$ eyes, nose, mouth
  - **Gene expression:** gene signatures, mutational signatures

NMF: iris example

```
library(NMF)
set.seed(42)
nmf_res <- nmf(as.matrix(iris[, 1:4]), 2, nrun = 20)
W <- basis(nmf_res); colnames(W) <- c("NMF1", "NMF2")
data.frame(W, Species = iris$Species) %>%
  ggplot(aes(NMF1, NMF2, fill = Species)) +
  geom_point(cex = 3, pch = 21)
```

NMF: iris example



WEJ LAB
RUTGERS HEALTH
Center for Data Science

NMF: iris example

- Setosa separates, driven mostly by one component
- All points lie in the positive quadrant (additive decomposition)
- `coef(nmf_res)` gives **H**: how each feature loads on each component

```
round(coef(nmf_res), 2)
```

```
##      Sepal.Length Sepal.Width Petal.Length Petal.Width
## [1,]         0.48        0.03         0.81        0.33
## [2,]         1.06        0.76         0.22        0.01
```

Non-linear methods: t-SNE and UMAP

- PCA, SVD, and NMF are **linear**
- Real data often lie on curved, **non-linear** structures
- **t-SNE** and **UMAP** preserve **local neighborhoods**: points close in high dimensions stay close in 2D
- Standard for single-cell data visualization

t-SNE

t-distributed Stochastic Neighbor Embedding (van der Maaten & Hinton, 2008):

1. Convert high-dimensional distances to neighbor probabilities
2. Find 2D points whose neighbor probabilities (t-distribution) match
3. Minimize the KL divergence between the two

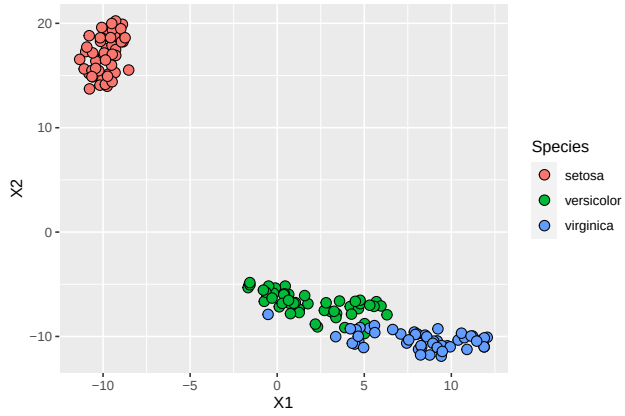
Key setting: **perplexity** ($\approx$ number of neighbors, typically 5-50)

WEJ LAB
RUTGERS HEALTH
Center for Data Science

t-SNE: iris example

```
library(Rtsne)
set.seed(1)
iris_u <- unique(iris)    # Rtsne requires no duplicate rows
tsne <- Rtsne(as.matrix(iris_u[, 1:4]), perplexity = 30)
data.frame(tsne$Y, Species = iris_u$Species) %>%
  ggplot(aes(X1, X2, fill = Species)) + geom_point(cex = 3, pch = 21)
```

t-SNE: iris example



UMAP

Uniform Manifold Approximation and Projection (McInnes et al., 2018):

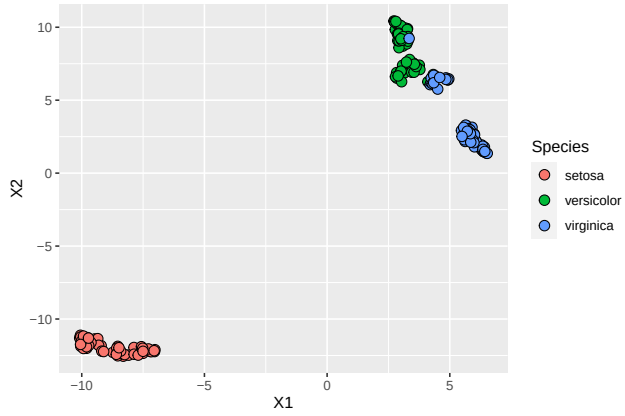
- Builds a weighted **nearest-neighbor graph** in high dimensions
- Finds a low-dimensional layout that preserves that graph
- Faster than t-SNE; often keeps more global structure
- Key settings: `n_neighbors`, `min_dist`

Explainer: https://pair-code.github.io/understanding-umap/

UMAP: iris example

```
library(uwot)
set.seed(1)
um <- umap(iris[, 1:4], n_neighbors = 15)
data.frame(um, Species = iris$Species) %>%
  ggplot(aes(X1, X2, fill = Species)) + geom_point(cex = 3, pch = 21)
```

UMAP: iris example

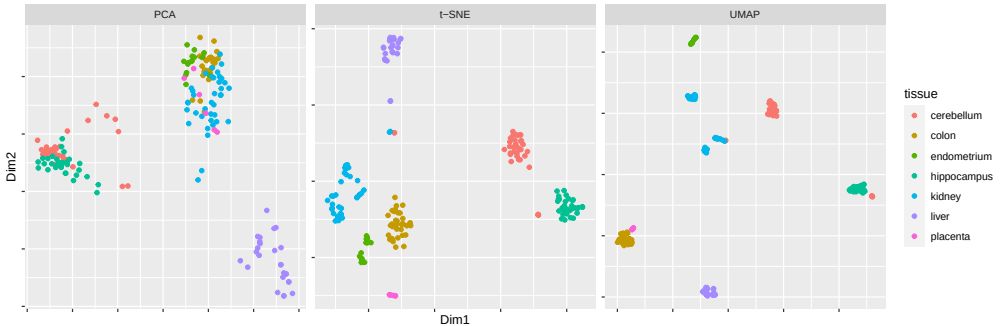


High-dimensional data: tissue gene expression

Iris has only 4 features. The real payoff comes with many features: 189 samples, 500 genes, 7 tissues. PCA first, then t-SNE/UMAP on the top 20 PCs:

```
library(dslabs); data("tissue_gene_expression")
xt <- sweep(tissue_gene_expression$x, 2, colMeans(tissue_gene_expression$x))
pcs <- prcomp(xt)$x[, 1:20]
set.seed(1)
emb <- list(PCA = pcs[, 1:2],
            `t-SNE` = Rtsne(pcs, perplexity = 30, pca = FALSE,
                            check_duplicates = FALSE)$Y,
            UMAP = umap(pcs, n_neighbors = 15))
```

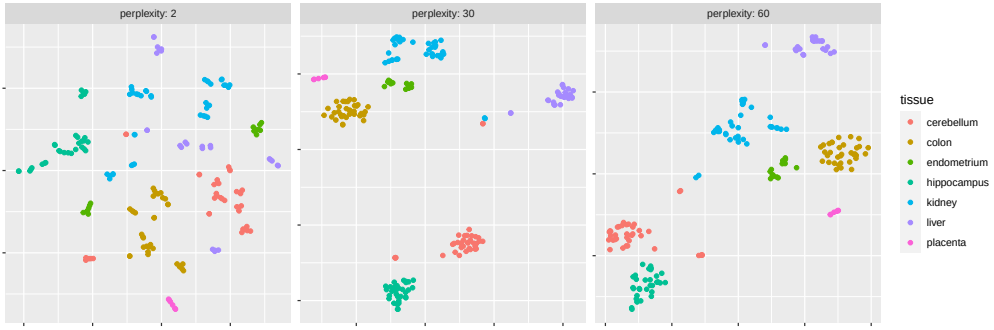
PCA vs. t-SNE vs. UMAP



PCA separates some tissues; t-SNE and UMAP separate all seven.

WEJ LAB

t-SNE: perplexity matters



Same data, three different pictures. Very small perplexity breaks tissues into fragments.

RUTGERS HEALTH
Center for Data Science

t-SNE and UMAP: cautions

- Results change with the random seed and tuning parameters
- Cluster **sizes** and **distances between clusters** are not meaningful
- Use for visualization, not as input features for downstream tests
- Common practice: run PCA first (e.g., top 20-50 PCs), then t-SNE/UMAP
- See Chari & Pachter (2023), "The specious art of single-cell genomics"

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Putting it together: single-cell RNA-seq

Standard workflow (what Seurat and Scanpy do under the hood):

1. Normalize and log-transform counts; pick highly variable genes
2. Scale genes; **PCA**; keep the top PCs
3. Build a **nearest-neighbor graph** in PC space
4. **Leiden** clustering on the graph
5. **UMAP** for visualization
6. Label clusters using marker genes

Single-cell data: 700 blood cells

PBMCs from Zheng et al. (2017), 765 highly variable genes, log-normalized (a subset distributed with Scanpy). Each cell also has a FACS-sorted cell-type label:

```
pbmc <- read.csv("../../python/data/pbmc68k_reduced.csv", check.names = FALSE)
cell_type <- pbmc$cell_type
x_sc <- as.matrix(pbmc[, -1])
dim(x_sc)
```

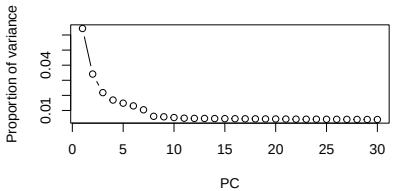
```
## [1] 700 765
```

```
sort(table(cell_type), decreasing = TRUE)
```

```
## cell_type
##                     Dendritic                CD14+ Monocyte
##                           240                           129
##                      CD19+ B               CD4+/CD25 T Reg
##                            95                            68
##             CD8+ Cytotoxic T CD8+/CD45RA+ Naive Cytotoxic
##                            54                            43
##                     CD56+ NK          CD4+/CD45RO+ Memory
##                            31                            19
```

Step 2: PCA

```
pc_sc <- prcomp(x_sc, scale. = TRUE, rank. = 30)
pve <- pc_sc$sdev^2 / sum(pc_sc$sdev^2)
plot(pve[1:30], type = "b", xlab = "PC", ylab = "Proportion of variance")
```



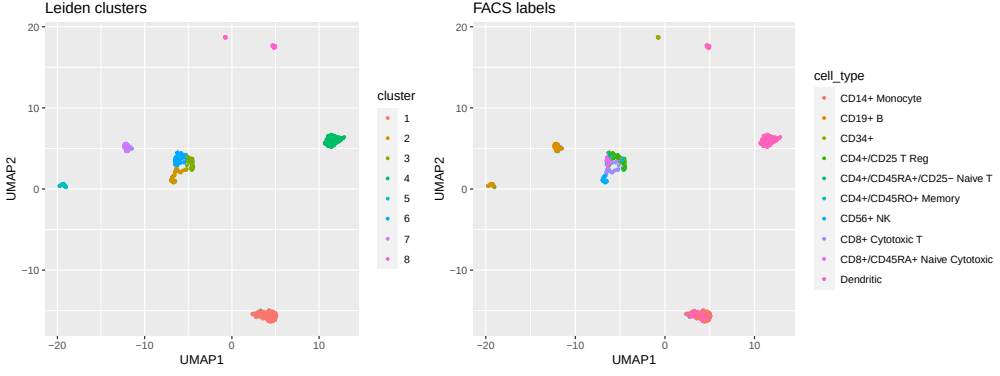
The scree plot flattens around PC 10-20; we keep 20.

Steps 3-5: graph, Leiden, UMAP

```
library(FNN); library(igraph)
pcs_sc <- pc_sc$x[, 1:20]
nn <- get.knn(pcs_sc, k = 15)$nn.index
g  <- graph_from_edgelist(cbind(rep(1:nrow(pcs_sc), 15), as.vector(nn)),
                          directed = FALSE) %>% simplify()
set.seed(1)
cluster <- factor(membership(cluster_leiden(g, objective_function = "modularity",
                                            resolution = 1, n_iterations = -1)))
um_sc <- umap(pcs_sc, n_neighbors = 15)
table(cluster)
```

```
## cluster
##   1   2   3   4   5   6   7   8
## 178  64  65 158  33  88  66  48
```

Single-cell UMAP



Clusters vs. cell-type labels

Labels abbreviated (Mono = CD14+ monocyte, DC = dendritic, nv = naive, mem = memory):

```
table(cluster, label = short[cell_type])
```

```
##         label
## cluster   B CD34 CD4mem CD4nv CD8 CD8nv  DC Mono  NK Treg
##       1   0    0      1     0   3     0  51  123   0    0
##       2   0    0      0     0  30     4   0    0  30    0
##       3   1    0     11     2   5     6   0    0   0   40
##       4   0    0      0     0   0     0 153    5   0    0
##       5  30    0      0     2   0     0   0    1   0    0
##       6   2    0      7     3  13    33   1    0   1   28
##       7  62    0      0     1   3     0   0    0   0    0
##       8   0   13      0     0   0     0  35    0   0    0
```

```
mclust::adjustedRandIndex(cluster, cell_type)
```

```
## [1] 0.5195588
```

Single-cell: lessons

- Monocytes, B cells, and CD34+ progenitors each map to their own clusters
- T-cell subtypes overlap: their FACS markers are proteins, and their RNA profiles are similar
- NK cells and cytotoxic T cells share a cluster (shared cytotoxic genes)
- "Dendritic" labels are spread across clusters: the bulk FACS labels are noisy
- Clusters are hypotheses; marker genes and biology confirm them

Comparing methods

Method	Type	Key constraint	Best for
PCA	Linear	Orthogonal components	General reduction; variance
SVD	Linear	U, V orthogonal	Low-rank approximation
NMF	Linear	Non-negative W, H	Parts-based, interpretable factors
t-SNE	Non-linear	Local neighborhoods	2D visualization
UMAP	Non-linear	Neighborhood graph	2D visualization; large data

WEJ LAB

Summary

- PCA rotates data so the first few components hold most of the variance
- Check the scree plot, read the loadings, and decide on scaling
- PCA = SVD of the centered data
- PCA is the first check for **batch effects**; ComBat corrects them
- NMF: additive, interpretable parts for non-negative data
- t-SNE/UMAP: great pictures of local structure, but don't over-read them
- Single-cell: PCA $\rightarrow$ kNN graph $\rightarrow$ Leiden $\rightarrow$ UMAP

Python companion

Python versions of these examples (`scikit-learn`, `umap-learn`, `scanpy`):

`python/week4/dimensionReduction.ipynb`

Session Info

```
sessionInfo()
```

```
## R version 4.3.3 (2024-02-29)
## Platform: x86_64-pc-linux-gnu (64-bit)
## Running under: Ubuntu 24.04.5 LTS
##
## Matrix products: default
## BLAS:   /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.12.0
## LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.12.0
##
## locale:
## [1] C
##
## time zone: America/Denver
## tzcode source: system (glibc)
##
## attached base packages:
## [1] parallel  stats     graphics  grDevices utils     datasets  methods
## [8] base
##
## other attached packages:
##  [1] igraph_1.6.0        FNN_1.1.4           dslabs_0.7.6
##  [4] doParallel_1.0.17   iterators_1.0.14    foreach_1.5.2
##  [7] NMF_0.27            cluster_2.1.6       rngtools_1.5.2
## [10] registry_0.5-1      sva_3.50.0          BiocParallel_1.36.0
## [13] genefilter_1.84.0   mgcv_1.9-1          nlme_3.1-164
## [16] bladderbatch_1.40.0 Biobase_2.62.0      BiocGenerics_0.48.1
## [19] MASS_7.3-60.0.1     uwot_0.1.16         Matrix_1.6-5
```